

✓ **Nishu_Qwen3_0.4B_Hinglish phone assistant (full fine-tune)**

Locate in Drive

Open in playground mode

New notebook in Drive

Open notebook Ctrl+O

Search notebooks in Drive

Upload notebook

Rename

Move

Move to trash

Save a copy in Drive

Save a copy as a GitHub Gist

Save a copy in GitHub

Save a copy with VS Code

Save Ctrl+S

Save and pin revision Ctrl+M S

Revision history

Notebook info

Download

Print Ctrl+P

set pe **full fine-tune** kiya jata hai (LoRA nahi), phir phone ke liye GGUF banaya jata hai.

U (T4 chalega; L4/A100 tez hain).

Google Drive (**MyDrive/nishu-qwen3**) mein save hota hai.

, Cell 1-5 chalo, phir Cell 6 dobara chalo. Wo apne aap last checkpoint se aage

1\train.jsonl aur test.jsonl (Cell 3 upload maangega).

```

s accelerate bitsandbytes
version__)
e(0) if torch.cuda.is_available() else "NONE -> Runtime > Change runtime type > GPU'
12.6/12.6 MB 69.2 MB/s eta 0:00:00
559.1/559.1 kB 29.3 MB/s eta 0:00:00
43.1/43.1 MB 18.7 MB/s eta 0:00:00
PROJECT_DIR = "/content/drive/MyDrive/nishu-qwen3"
DATA_DIR = f"{PROJECT_DIR}/data"
CKPT_DIR = f"{PROJECT_DIR}/checkpoints"
FINAL_DIR = f"{PROJECT_DIR}/final-model"
GGUF_DIR = f"{PROJECT_DIR}/gguf"
for d in (DATA_DIR, CKPT_DIR, FINAL_DIR, GGUF_DIR):
    os.makedirs(d, exist_ok=True)
print("Project folder:", PROJECT_DIR)

```

Mounted at /content/drive
Project folder: /content/drive/MyDrive/nishu-qwen3

✓ **Cell 3: Data upload (sirf pehli baar)**

```

import shutil
needed = ["train.jsonl", "test.jsonl"]
missing = [f for f in needed if not os.path.exists(f"{DATA_DIR}/{f}")]
if missing:
    from google.colab import files
    print("Ye files upload karo (D:\\research_llm\\data\\v2\\final\\ se):", missing)
    for name in files.upload():
        target = "train.jsonl" if name.startswith("train") else "test.jsonl" if name.startswith("test") else name
        shutil.move(name, f"{DATA_DIR}/{target}")
print({f: os.path.exists(f"{DATA_DIR}/{f}") for f in needed})

```

Ye files upload karo (D:\\research_llm\\data\\v2\\final\\ se): ['train.jsonl', 'test.jsonl']

Choose Files | No file chosen | Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving stats.json to stats.json

Saving test.jsonl to test.jsonl

Saving train.jsonl to train.jsonl

✓ **Cell 4: Config**

LEARNING_RATE = 3e-5 full fine-tuning ke liye rakha hai (LoRA wala 2e-4 full FT pe bahut tez hai aur model ki purani English/tool skills bhula sakta hai). Agar training ke baad model Hinglish/tools theek se nahi seekhta, isse 5e-5 karke dobara try karna.

```

MODEL_ID      = "Qwen/Qwen3-0.6B"
MAX_LEN       = 1024  # sabse lamba sample ~811 tokens hai
LEARNING_RATE = 3e-5
NUM_EPOCHS    = 3
PER_DEVICE_BATCH = 2
GRAD_ACCUM    = 16    # effective batch = 32
SAVE_STEPS    = 100
SEED          = 42

```

✓ Cell 5: Data ko training samples mein badlo

Har assistant turn ek alag sample banta hai, aur uska prompt bilkul waise hi render hota hai jaise phone pe hoga (khali `<think>` `</think>` ke saath, thinking off). Loss **sirf assistant ke jawab** pe lagta hai, system prompt / user / tool result pe nahi.

```

import json
from datasets import Dataset
from transformers import AutoTokenizer

tok = AutoTokenizer.from_pretrained(MODEL_ID)

def render(msgs, gen_prompt=False):
    return tok.apply_chat_template(msgs, tokenize=False, add_generation_prompt=gen_prompt, enable_thinking=False)

def conversation_to_samples(msgs):
    out = []
    for i, m in enumerate(msgs):
        if m["role"] != "assistant":
            continue
        prompt = render(msgs[:i], gen_prompt=True)
        full = render(msgs[:i + 1])
        if not full.startswith(prompt):
            raise ValueError("prompt/full render mismatch")
        p_ids = tok(prompt, add_special_tokens=False)["input_ids"]
        c_ids = tok(full[len(prompt):], add_special_tokens=False)["input_ids"]
        ids = p_ids + c_ids
        if len(ids) <= MAX_LEN:
            out.append({"input_ids": ids, "attention_mask": [1] * len(ids), "labels": [-100] * len(p_ids) + c_ids})
    return out

def load_samples(path):
    rows = [json.loads(l) for l in open(path, encoding="utf-8") if l.strip()]
    samples = [s for r in rows for s in conversation_to_samples(r["messages"])]
    return rows, Dataset.from_list(samples)

train_rows, train_ds = load_samples(f"{DATA_DIR}/train.jsonl")
test_rows, eval_ds = load_samples(f"{DATA_DIR}/test.jsonl")
SYSTEM_PROMPT = train_rows[0]["messages"][0]["content"]
print(f"train: {len(train_rows)} conversations -> {len(train_ds)} samples")
print(f"test: {len(test_rows)} conversations -> {len(eval_ds)} samples")
print("\nSystem prompt (phone app mein bhi yahi jayega):\n" + SYSTEM_PROMPT)

```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and fast
 WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to

```

config.json: 100%                               726/726 [00:00<00:00, 49.9kB/s]

tokenizer_config.json: 100%                       9.73k/9.73k [00:00<00:00, 425kB/s]

vocab.json: 100%                                2.78M/2.78M [00:00<00:00, 31.8MB/s]

merges.txt: 100%                                1.67M/1.67M [00:00<00:00, 25.6MB/s]

tokenizer.json: reconstructing file: 100%          11.4MB / 11.4MB, 1.12MB/s

tokenizer.json: downloading bytes:                 3.40MB, 332kB/s

train: 6212 conversations -> 7736 samples
test: 320 conversations -> 403 samples

```

System prompt (phone app mein bhi yahi jayega):

Tum Nishu ho, ek helpful assistant jo Hinglish aur English dono mein baat karta hai. User jis bhasha mein pooche, usi bhasha

```

Tools (jab zaroorat ho, sirf <tool_call>{"name": "...", "arguments": {...}}</tool_call> likho):
calculator(expression:str) | + - * / % ** ( )
unit_converter(value:num, from_unit, to_unit) | km miles kg lbs celsius fahrenheit meters feet liters gallons
date_time(query:today|day_of_week|+N days|-N days)
set_alarm(hour:int, minute:int, period?:morning|afternoon|evening|night, day?:today|tomorrow|day_after_tomorrow|mon..sun, re
set_timer(hours?:int, minutes?:int, seconds?:int, label?:str)
device_control(target:flashlight|wifi|bluetooth|silent|vibrate, state:on|off|toggle)
open_app(app_name:str)
make_call(contact?:str, number?:str) | exactly one
send_message(contact:str, text:str, app:whatsapp|sms|default)

```

Cell 6: Training (fresh ya resume, dono yahi cell)

Checkpoint Drive pe mila toh wahin se aage chalega. T4 pe ~1.5-2.5 ghante lag sakte hain.

```
import math, torch
from transformers import AutoModelForCausalLM, Trainer, TrainingArguments, DataCollatorForSeq2Seq
from transformers.trainer_utils import get_last_checkpoint

use_bf16 = torch.cuda.is_bf16_supported()
# Weights fp32 mein rakhte hain (sirf compute bf16/fp16 mein), taaki chhote updates round-off mein na khoyein.
model = AutoModelForCausalLM.from_pretrained(MODEL_ID, dtype=torch.float32)
model.config.use_cache = False

steps_per_epoch = math.ceil(len(train_ds) / (PER_DEVICE_BATCH * GRAD_ACCUM))
warmup_steps = max(1, int(0.05 * steps_per_epoch * NUM_EPOCHS))

args = TrainingArguments(
    output_dir=CKPT_DIR,
    per_device_train_batch_size=PER_DEVICE_BATCH,
    per_device_eval_batch_size=PER_DEVICE_BATCH,
    gradient_accumulation_steps=GRAD_ACCUM,
    learning_rate=LEARNING_RATE,
    lr_scheduler_type="cosine",
    warmup_steps=warmup_steps,
    num_train_epochs=NUM_EPOCHS,
    weight_decay=0.01,
    max_grad_norm=1.0,
    bf16=use_bf16,
    fp16=not use_bf16,
    optim="paged_adamw_8bit",
    gradient_checkpointing=True,
    gradient_checkpointing_kwargs={"use_reentrant": False},
    logging_steps=10,
    eval_strategy="epoch",
    save_strategy="steps",
    save_steps=SAVE_STEPS,
    save_total_limit=2,
    seed=SEED,
    report_to="none",
    remove_unused_columns=False,
)

trainer = Trainer(
    model=model,
    args=args,
    train_dataset=train_ds,
    eval_dataset=eval_ds,
    data_collator=DataCollatorForSeq2Seq(tok, padding=True, label_pad_token_id=-100, pad_to_multiple_of=8),
)

last = get_last_checkpoint(CKPT_DIR) if os.path.isdir(CKPT_DIR) else None
print(f'{"Resuming from " + last if last else "Fresh training"} | {steps_per_epoch} steps/epoch x {NUM_EPOCHS} epochs | '
      f"precision: {'bf16' if use_bf16 else 'fp16'}")
trainer.train(resume_from_checkpoint=last)

trainer.model.to(torch.bfloat16).save_pretrained(FINAL_DIR, safe_serialization=True)
tok.save_pretrained(FINAL_DIR)
print("Final model saved:", FINAL_DIR)
```

model.safetensors: reconstructing file: 100%

1.50GB / 1.50GB, 91.5MB/s

model.safetensors: downloading bytes:

1.26GB, 78.4MB/s

Loading weights: 100%

311/311 [00:03<00:00, 116.08it/s]

generation_config.json: 100%

239/239 [00:00<00:00, 5.33kB/s]

Resuming from /content/drive/MyDrive/nishu-qwen3/checkpoints/checkpoint-700 | 242 steps/epoch x 3 epochs | precision: bf16
[transformers] There were missing keys in the checkpoint model loaded: ['lm_head.weight'].

[726/726 17:18, Epoch 3/3]

Epoch	Training Loss	Validation Loss
3	0.959307	1.855549

Writing model shards: 100%

1/1 [00:24<00:00, 24.78s/it]

Writing model shards: 100%

1/1 [00:06<00:00, 6.24s/it]

Final model saved: /content/drive/MyDrive/nishu-qwen3/final-model

Cell 7: Quick test + measured accuracy

Pehle kuch haath ke sawaal, phir test set pe 3 numbers:

- **tool name accuracy:** sahi tool chuna?
- **exact arguments:** arguments bilkul sahi?
- **false trigger rate:** normal baat-cheet pe galti se tool chala diya? (1% se kam hona chahiye)

```

import re, torch
from transformers import AutoModelForCausalLM

dtype = torch.bfloat16 if torch.cuda.is_bf16_supported() else torch.float16
eval_model = AutoModelForCausalLM.from_pretrained(FINAL_DIR, dtype=dtype).to("cuda").eval()

def generate(msgs, max_new_tokens=200):
    text = tok.apply_chat_template(msgs, tokenize=False, add_generation_prompt=True, enable_thinking=False)
    ids = tok(text, return_tensors="pt").to("cuda")
    with torch.no_grad():
        out = eval_model.generate(**ids, max_new_tokens=max_new_tokens, do_sample=False,
                                   pad_token_id=tok.pad_token_id)
    return tok.decode(out[0][ids["input_ids"].shape[1]:], skip_special_tokens=False).replace("<|im_end|>", "").strip()

battery = [
    "Tum kaun ho?",
    "Photosynthesis simple words me samjhao",
    "Kal subah sawa 6 baje ka alarm laga do",
    "Mummy ko WhatsApp karo ki main 8 baje tak aa jaunga",
    "1250 rupaye pe 18% GST kitna hoga?",
    "Timer aur stopwatch mein kya farak hai?",
    "Explain what inflation is in simple terms.",
    "Translate to Hinglish: The meeting has been postponed to Monday.",
]
for q in battery:
    print(f"\n>>> {q}\n{generate([{'role': 'system', 'content': SYSTEM_PROMPT}, {'role': 'user', 'content': q}])}")

def parse_call(text):
    m = re.search(r"<tool_call>\s*(\{.*?\})\s*</tool_call>", text, re.S)
    if not m:
        return None
    try:
        c = json.loads(m.group(1))
        args = c.get("arguments", {})
        return c.get("name"), json.loads(args) if isinstance(args, str) else args
    except json.JSONDecodeError:
        return ("<unparseable>", {})

import random
from collections import Counter
def first_assistant(row):
    return next(m for m in row["messages"] if m["role"] == "assistant")
tool_rows = [r for r in test_rows if first_assistant(r).get("tool_calls")]
chat_rows = [r for r in test_rows if not first_assistant(r).get("tool_calls")]
N_CHAT = 150 # test file is grouped by category, so sample chat rows randomly; all tool rows are used
chat_rows = random.Random(0).sample(chat_rows, min(N_CHAT, len(chat_rows)))
print(f"\nEvaluating {len(tool_rows)} tool-call rows + {len(chat_rows)} chat rows ...")

tool_total = tool_name_ok = tool_args_ok = chat_total = false_triggers = 0
per_tool = Counter(); per_tool_ok = Counter()
for row in tool_rows + chat_rows:
    msgs = row["messages"]
    i = next(k for k, m in enumerate(msgs) if m["role"] == "assistant")
    pred = parse_call(generate(msgs[:i], max_new_tokens=96))
    gold = msgs[i]
    if gold.get("tool_calls"):
        f = gold["tool_calls"][0]["function"]
        g_args = json.loads(f["arguments"]) if isinstance(f["arguments"], str) else f["arguments"]
        tool_total += 1
        tool_name_ok += bool(pred and pred[0] == f["name"])
        tool_args_ok += bool(pred and pred[0] == f["name"] and pred[1] == g_args)
        per_tool[f["name"]] += 1
        per_tool_ok[f["name"]] += bool(pred and pred[0] == f["name"] and pred[1] == g_args)
    else:
        chat_total += 1
        false_triggers += pred is not None

print(f"\nTool calls : name accuracy {tool_name_ok}/{tool_total} ({100*tool_name_ok/max(1,tool_total):.0f}%), "
      f"exact arguments {tool_args_ok}/{tool_total} ({100*tool_args_ok/max(1,tool_total):.0f}%)"
      f"Normal chat : false tool triggers {false_triggers}/{chat_total} ({100*false_triggers/max(1,chat_total):.1f}%)"
      f"Per tool (exact args): " + ", ".join(f"{t} {per_tool_ok[t]}/{n}" for t, n in per_tool.most_common()))
del eval_model; torch.cuda.empty_cache()

```

Loading weights: 100%

310/310 [00:00<00:00, 1868.09it/s]

Cell 8: GGUF banao (phone ke liye)

>>> Tum kaun ho?

Main Nishu hoon, ek Hinglish aur English dono baat karne wale smart assistant.
 Pehle F16 GGUF (conversion check), phir Q4_K_M (phone wali file). Naye runtime mein bhi akela ye cell kaafi hai, agar

final-model

Drive pe hai. Photosynthesis simple words me samjhao

Photosynthesis wo process hai jisme paudhe din mein paani, sunlight aur carbon dioxide ko uska aadha bana kar uska aadha bar

```
import os, json, subprocess

if not os.path.exists("/content/drive/MyDrive"):
    from google.colab import drive
    drive.mount('/content/drive')
PROJECT_DIR = "/content/drive/MyDrive/nishu-qwen3"
FINAL_DIR, GGUF_DIR = f"{PROJECT_DIR}/final-model", f"{PROJECT_DIR}/gguf"
os.makedirs(GGUF_DIR, exist_ok=True)
f16_path, q4_path = f"{GGUF_DIR}/nishu-f16.gguf", f"{GGUF_DIR}/nishu-Q4_K_M.gguf"
assert os.path.exists(f"{FINAL_DIR}/config.json"), "final-model nahi mila - pehle Cell 6 chalao"

# Patch tokenizer_config.json to prevent 'list' object has no attribute 'keys' error
tok_config_path = f"{FINAL_DIR}/tokenizer_config.json"
if os.path.exists(tok_config_path):
    try:
        with open(tok_config_path, "r", encoding="utf-8") as f:
            cfg = json.load(f)
            if "extra_special_tokens" in cfg and isinstance(cfg["extra_special_tokens"], list):
                print("Patching tokenizer_config.json (extra_special_tokens was a list)...")
                cfg["extra_special_tokens"] = {}
                with open(tok_config_path, "w", encoding="utf-8") as f:
                    json.dump(cfg, f, indent=2, ensure_ascii=False)
    except Exception as e:
        print("Tokenizer patch warning:", e)

quantize_bin = "/content/llama.cpp/build/bin/llama-quantize"
if not os.path.exists(quantize_bin):
    print("llama.cpp build ho raha hai (5-10 min)...")
    if not os.path.exists("/content/llama.cpp"):
        subprocess.run(["git", "clone", "--depth", "1", "https://github.com/ggml-org/llama.cpp", "/content/llama.cpp"], check=True)
    subprocess.run(["cmake", "-B", "/content/llama.cpp/build", "-S", "/content/llama.cpp", "-DGGML_CUDA=OFF"], check=True)
    # -j 2: free Colab ki kam RAM mein unlimited parallel build compiler ko crash kar deta hai
    subprocess.run(["cmake", "--build", "/content/llama.cpp/build", "--config", "Release", "-j", "2"], check=True)
    subprocess.run(["pip", "install", "-q", "-r", "/content/llama.cpp/requirements.txt"], check=True)

def run(cmd, what):
    print(f"\n{what}...")
    r = subprocess.run(cmd, capture_output=True, text=True)
    if r.returncode != 0:
        print(r.stderr[-3000:])
        raise SystemExit(f"{what} FAILED - upar ka error paste karo")

run(["python", "/content/llama.cpp/convert_hf_to_gguf.py", FINAL_DIR, "--outfile", f16_path, "--outtype", "f16"], "GGUF F16")
run([quantize_bin, f16_path, q4_path, "Q4_K_M"], "Q4_K_M quantization")
for p in (f16_path, q4_path):
    print(f"{os.path.basename(p)}: {os.path.getsize(p)/2**20:.0f} MB")

system_prompt = json.loads(open(f"{PROJECT_DIR}/data/train.jsonl", encoding="utf-8").readline()[0]["content"])
open(f"{GGUF_DIR}/system_prompt.txt", "w", encoding="utf-8").write(system_prompt)
print("System prompt saved:", f"{GGUF_DIR}/system_prompt.txt (phone app ko yahi bhejna hai)")
```

Patching tokenizer_config.json (extra_special_tokens was a list)...

GGUF F16 conversion...

Q4_K_M quantization...

nishu-f16.gguf: 1143 MB

nishu-Q4_K_M.gguf: 378 MB

System prompt saved: /content/drive/MyDrive/nishu-qwen3/gguf/system_prompt.txt (phone app ko yahi bhejna hai)